Name: Palak Chanchlani
Class: D20A
Roll No: 12

EXPERIMENT 10

Aim:
To explore the cryptocurrency landscape using real-world data and blockchain concepts.

Theory:
This study analyzes the crypto market using Chainlink (LINK) as the focus asset. Data was collected from:

- CoinGecko API – Historical price, market cap, and volume (used in Python).
- CoinMarketCap – Price charts and market ranking.
- Crypto.com – Market stats and news updates.

Cryptocurrency Landscape:

- **Definition:** Digital assets based on blockchain enabling secure, decentralized transactions.
- **Types:**
  - Store of Value – Bitcoin, Litecoin
  - Utility Tokens – Ethereum, Chainlink
  - Stablecoins – USDT, USDC
  - Governance Tokens – UNI, AAVE
- **Volatility:** Prices fluctuate due to market trends and regulations.
- **Importance:** Data analysis helps track performance and trends.
- **Advancements:** DeFi, NFTs, Layer-2 scaling, cross-chain systems.

Data Collection & Analysis:
Using CoinGecko API and Pandas:

- Daily Returns – Measures price changes
- Moving Averages (MA7, MA30) – Trend indicators
- Cumulative Returns – Overall growth
- Rolling Volatility – Risk analysis

Visualization:
A dashboard (Matplotlib) showing:

- Price Trend
- Price vs Volume
- Daily Returns Distribution
- Moving Averages
- Cumulative Returns
- Rolling Volatility

Technological Analysis:

- Consensus: PoW, PoS
- Scalability: Layer-2, sharding
- Interoperability: Chainlink CCIP, Polkadot, Cosmos
- Privacy: zk-SNARKs, ring signatures

Chainlink focuses on staking, oracle improvements, and cross-chain capabilities.

Market Trends:

- Increased crypto adoption by businesses
- Growth in crypto wallets
- Higher network activity
- Integration with fintech
- Evolving regulations

Utilizing Api for analysis and visualization :
Code:

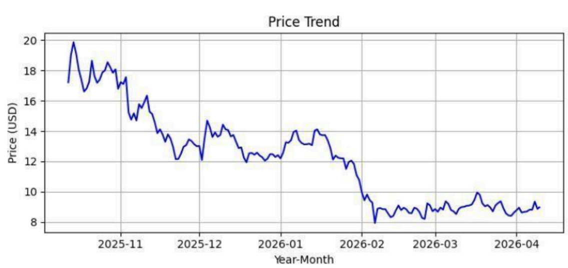
```
import requests

import matplotlib.pyplot as plt

import pandas as pd


# ------- Fetch Data -------

def get_historical_data(coin_id, days='180', currency='usd'):

    url = f'https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart'

    params = {'vs_currency': currency, 'days': days}

    response = requests.get(url, params=params)

    return response.json()


# ---------- Dashboard Function
```

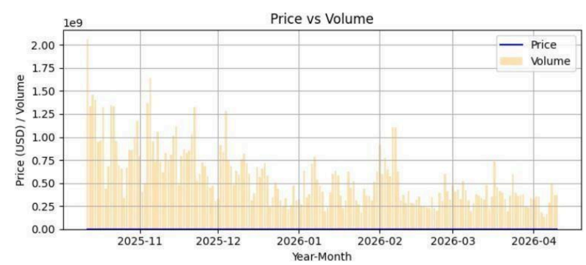
```
data = get_historical_data(coin_id, days)

    # Create DataFrame
    prices_df = pd.DataFrame(data['prices'], columns=['timestamp', 'price'])
    volumes_df = pd.DataFrame(data['total_volumes'], columns=['timestamp', 'volume'])

    prices_df['timestamp'] = pd.to_datetime(prices_df['timestamp'], unit='ms')
    volumes_df['timestamp'] = pd.to_datetime(volumes_df['timestamp'], unit='ms')

    df = pd.merge(prices_df, volumes_df, on='timestamp')
    df.set_index('timestamp', inplace=True)

    # Calculate extra metrics
    df['returns'] = df['price'].pct_change()
    df['MA7'] = df['price'].rolling(window=7).mean()
    df['MA30'] = df['price'].rolling(window=30).mean()
    df['cumulative'] = (1 + df['returns']).cumprod()
    df['volatility'] = df['returns'].rolling(window=7).std() * 100

    # ---------- Plot Dashboard ----------
    fig, axes = plt.subplots(3, 2, figsize=(16, 12))
    fig.suptitle(f'{coin_id.capitalize()} Dashboard (Last {days} Days)', fontsize=16, fontweight='bold')

    # 1. Price Trend
    axes[0, 0].plot(df.index, df['price'], color='blue')
    axes[0, 0].set_title('Price Trend')
    axes[0, 0].set_ylabel('Price (USD)')
    axes[0, 0].set_xlabel('Year-Month') # Add xlabel
    axes[0, 0].grid(True)
```

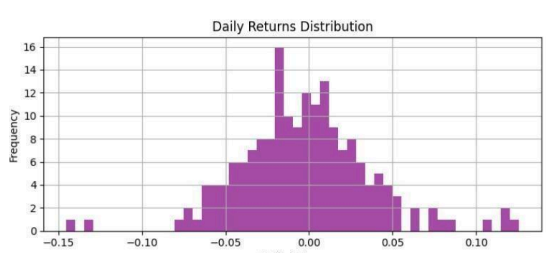
```python
# 2. Price vs Volume
axes[0, 1].plot(df.index, df['price'], color='blue', label='Price')
axes[0, 1].bar(df.index, df['volume'], color='orange', alpha=0.3, label='Volume')
axes[0, 1].set_title('Price vs Volume')
axes[0, 1].set_xlabel('Year-Month') # Add xlabel
axes[0, 1].set_ylabel('Price (USD) / Volume') # Add ylabel
axes[0, 1].legend()
axes[0, 1].grid(True)

# 3. Daily Returns Distribution
axes[1, 0].hist(df['returns'].dropna(), bins=50, alpha=0.7, color='purple')
axes[1, 0].set_title('Daily Returns Distribution')
axes[1, 0].set_xlabel('Daily Return')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].grid(True)

# 4. Moving Averages
axes[1, 1].plot(df.index, df['price'], color='lightgray', label='Price')
axes[1, 1].plot(df.index, df['MA7'], color='blue', label='7-day MA')
axes[1, 1].plot(df.index, df['MA30'], color='red', label='30-day MA')
axes[1, 1].set_title('Moving Averages')
axes[1,  1].set_xlabel('Year-Month')
axes[1, 1].set_ylabel('Price (USD)')
axes[1, 1].legend()
axes[1, 1].grid(True)

# 5. Cumulative Returns
axes[2, 0].plot(df.index, df['cumulative'], color='green')
axes[2, 0].set_title('Cumulative Returns')
axes[2, 0].set_ylabel('Growth Factor')
```

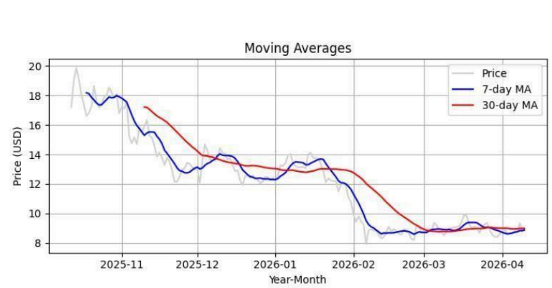
```
    axes[2, 0].set_xlabel('Year-Month')
    axes[2, 0].grid(True)

    # 6. Volatility
    axes[2, 1].plot(df.index, df['volatility'], color='orange')
    axes[2, 1].set_title('Rolling Volatility (7-Day)')
    axes[2, 1].set_ylabel('Volatility (%)')
    axes[2, 1].set_xlabel('Year-Month')
    axes[2, 1].grid(True)

    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.subplots_adjust(hspace=0.5, wspace=0.3) # Adjust vertical and horizontal spacing
    plt.show()
# ---------- Run Dashboard
----------chainlink_dashboard('180')
```

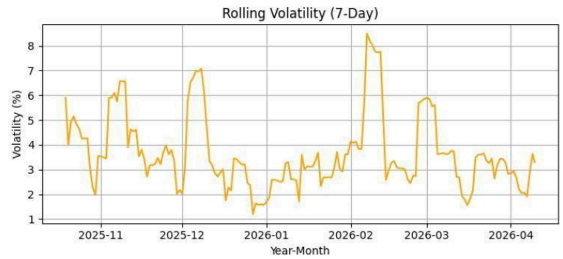
Output:



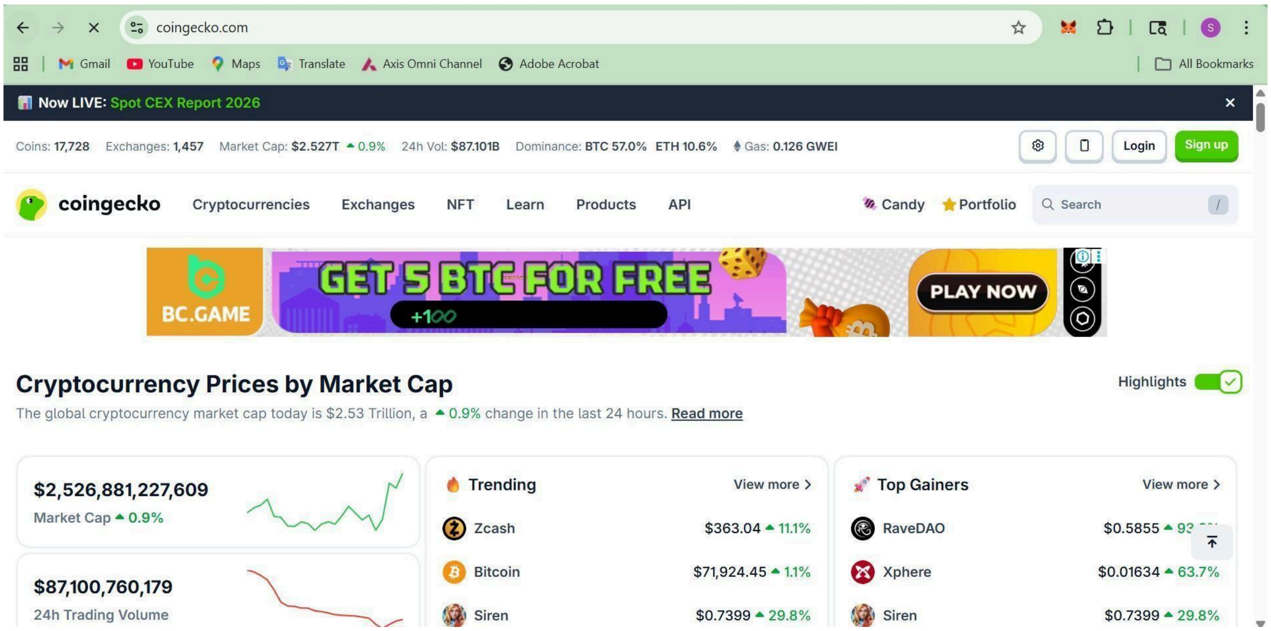


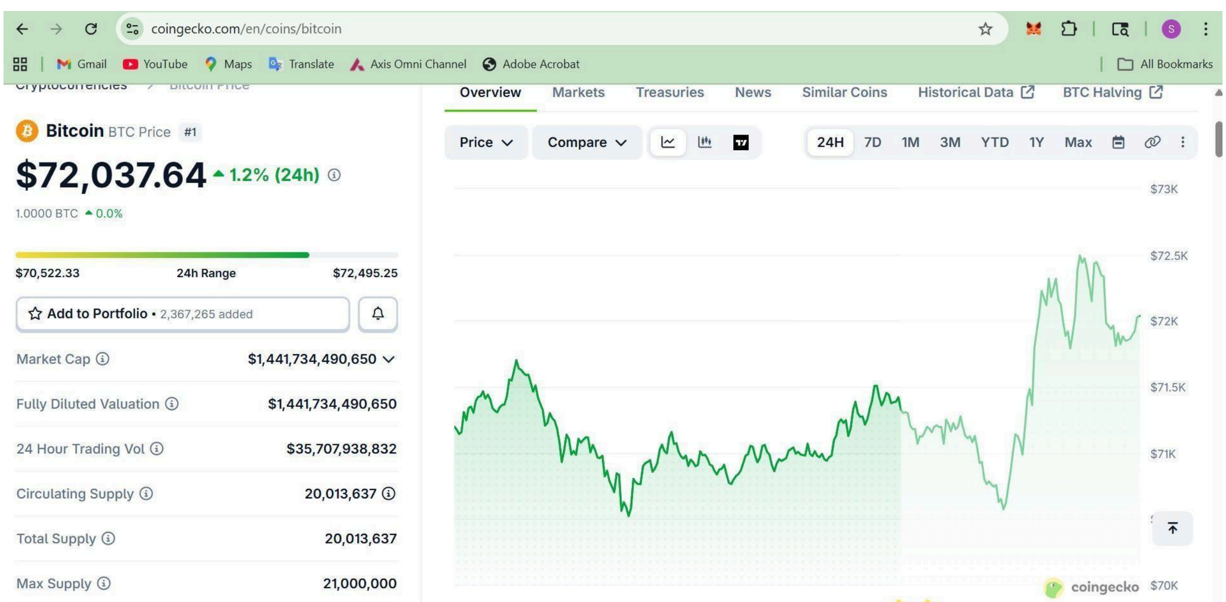




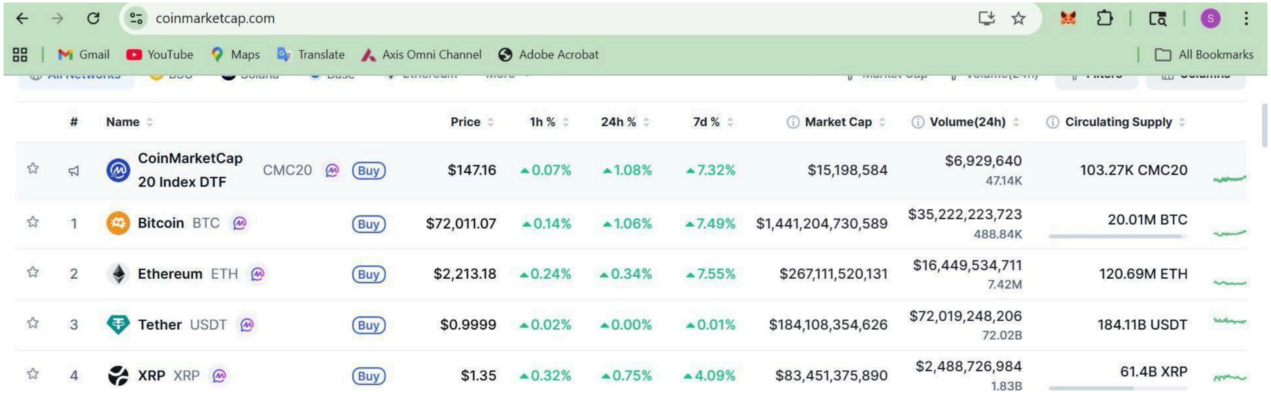


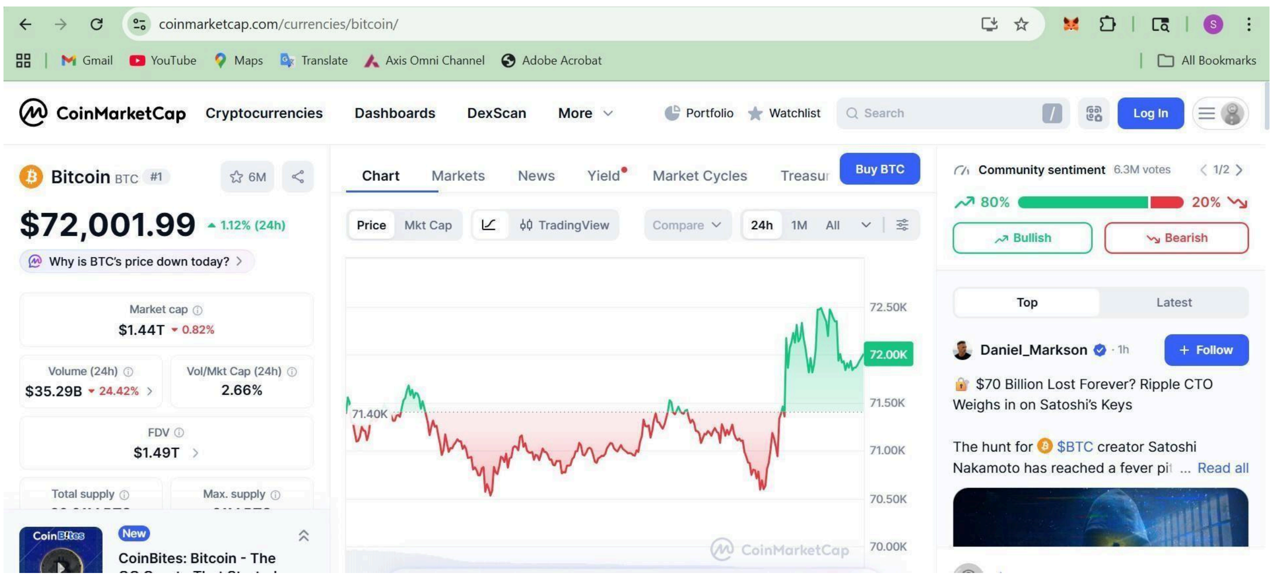
Coingecko Website





CoinMarketCap website





Conclusion:

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.